

Handling Test Smells in Python: Results from a Mixed-Method Study

Daniel David Fernandes
Federal University of Bahia (UFBA)
Salvador, Brazil
daniel.david@ufba.br

Ivan Machado
Federal University of Bahia (UFBA)
Salvador, Brazil
ivan.machado@ufba.br

Rita S. P. Maciel
Federal University of Bahia (UFBA)
Salvador, Brazil
rita.suzana@ufba.br

ABSTRACT

Software testing is an activity in the software development process that looks for defects. Automated testing is composed of code that allows run software testing scenarios more quickly, avoiding manual rework. However, testers are likely to employ practices that might negatively impact test quality regarding maintainability, understandability, and effectiveness when writing test code. Such bad practices are also known as test smells. Although test smells are a language-independent concept, different programming languages could present different occurrence standards. Therefore, studies in one programming language may not be generalizable. This study aims to investigate how test smells occurrence in Python test files. Python became the most widely used programming language globally in 2020. However, most research on test code quality only considers the Java language. To accomplish our goals, we built a dataset with 5,303 test files from 90 Python projects collected from GitHub repositories to understand and analyze strategies for handling test smells in Python. This analysis allowed us to propose four new test smells, discussing their potential problems. We carried out a preliminary evaluation with 40 Python developers to validate their thoughts on the proposed test smells. These results are part of an ongoing research project aiming to propose a foundation to better support automation tests in Python.

CCS CONCEPTS

• **Software and its engineering** → **Software verification and validation.**

KEYWORDS

Test Smells, Automated Software Testing, Test Code Quality, Python

ACM Reference Format:

Daniel David Fernandes, Ivan Machado, and Rita S. P. Maciel. 2021. Handling Test Smells in Python: Results from a Mixed-Method Study. In *Brazilian Symposium on Software Engineering (SBES '21)*, September 27–October 1, 2021, Joinville, Brazil. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3474624.3477066>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SBES '21, September 27–October 1, 2021, Joinville, Brazil

© 2021 Association for Computing Machinery.
ACM ISBN 978-1-4503-9061-3/21/09...\$15.00
<https://doi.org/10.1145/3474624.3477066>

1 INTRODUCTION

Software testing is the activity that looks for software faults. It is considered essential for software quality assurance in almost every software development methodology [12]. Software testing can be performed manually or automated. In manual testing, testers assume the role of an end-user interacting with the running software to verify its behavior and find possible faults [11]. In automated testing, test code scripts are developed using frameworks or testing tools and then run to test the software behavior. As software evolves, tests need to be rerun to ensure that both old and new functionality works as expected. Therefore, automated testing saves repetitive manual rework and may decrease the time to execute test code.

Automated testing could bring several benefits when compared to manual testing, such as repeatability, reduced effort, faster execution, and consequently reduced costs [25]. However, if not implemented correctly, following good practices, automated testing can lead to problems and, in some cases, become less effective than manual testing in detecting faults [2].

Bad practices in software test coding can be detrimental to understanding and effectiveness of automated testing. These bad practices are known as test smells [7] and may negatively affect test code, as they lead to poor test comprehensibility, maintainability, and effectiveness [27].

Python has grown due to its versatile syntax [14]. Python is a high-level and multi-paradigm language (imperative, functional, and object-oriented) that can be interpreted, scripted, and enable dynamic and strong typing [22]. Recent research shows that Python is the fastest-growing programming language among software developers and has already taken the position of the most popular programming language in the world¹. Researchers have investigated the growing demand for the Python programming language and positive aspects of software development using this language [14].

Although the Python language's growing use, most current research works, and tools to support test smells are related to the Java language, test smell is not academically explored considering the Python context yet [1]. Therefore, automated testing in Python lacks test quality support, which can lead to the negative impacts caused by the presence of test smells as well as problems like tests non-determinism and low effectiveness in finding software faults [8, 21].

This paper presents preliminary results of an ongoing research project that aims to provide a rationale for test smells handling and test automated support for Python. Toward this goal achievement,

¹<https://spectrum.ieee.org/at-work/tech-careers/top-programming-language-2020>

we built and made publicly available a Python test files dataset² from GitHub repositories and investigated the occurrence standard of 20 test smells in Python projects to understand strategies to handle test smells in Python. As a result of this investigation, we proposed four new test smells validated by 40 software testing professionals. We aim to support Python developers in building and maintaining higher quality test code, avoiding bad practices reported in the literature. By sharing these results, we hope for future contributions and improvements in this research work.

2 TEST SMELLS

When developing software, some fundamental design principles may be violated, negatively impacting the software design quality. Therefore, the Smells concept in software arose to describe these anomalies [10]. During software implementation and evolution phases, test code tends to grow as new test cases are needed to be associated with new software functionalities. Developers tend to focus more on the production code rather than test code. However, test code quality could directly impact the quality of source code [24].

Automated tests can save time and effort because they can be rerun when software evolves to validate behavior changes among system components. Several works have analyzed the impact of the presence of test smells in test code to improve production code quality by using automated tests more efficiently [3, 23, 30]. It is essential to know about and to avoid them, keeping a standard quality of how test cases should be implemented, organized, and interact with each other [18].

Although developers' perception of test smells has shown that developers recognize test smells as potential problems and prioritize some test smells as most critical [24], many developers still do not know some test smells [11, 15]. In addition, empirical evaluations pointed out that different contexts such as distinct programming languages and their developers' perspectives may lead to diverse conclusions [18, 20, 23].

There is research on test code quality in Python [26, 31] and the impacts of bad coding practices on production code (code smells) in Python projects [4, 28]. However, test code quality considering test smells were not investigated in Python projects yet.

In summary, studies on test smells are predominantly for Java, both in theory (empirical evaluations) and practice (tools) [1]. This state-of-the-art analysis reinforces the interest in investigating test smells from a Python programming language perspective due to its increasing use and the lack of studies to investigate test smells in Python.

3 RELATED WORK

Deursen et al. [7] introduced the concept of test smells and presented a catalog of test smells related to bad smells. They proposed and discussed twelve types of test smells, presenting refactoring strategies.

Meszaros et al. [17] discussed the effects of existing and proposed practices on test code with some refactoring strategies. In

addition, they presented test coding patterns and principles from the literature aimed at increasing the quality of automated testing.

Greiler et al. [13] presented techniques to identify six test smells. The authors implemented a static analysis tool called TestHound, which provides reports on test code for refactoring the test smells. The authors evaluated the TestHound tool through three industrial case studies. It shows that developers find that the tool helps them understand and adjust test code.

Peruma et al. [18] proposed twelve new test smells and surveyed over 40 developers who confirmed the proposed test smells, indicating bad programming practices. Additionally, they perform an empirical study on the occurrences and distribution of the proposed smells on 656 open-source Android applications in the java programming language.

De Bleser et al. [6] analyzed six test smells in 164 open-source Scala projects hosted on GitHub and created SoCRATES, a test smell automated detection tool for Scala. In addition, they evaluated the perception of test smells with 14 Scala developers through interviews.

Although these studies are relevant, still missing research on test code quality regarding test smells in Python and a tool to support test smells in Python. In this paper, we analyze the occurrence standard of 20 test smells already proposed in Python projects. In this analysis, we proposed four new test smells, which test professionals agreed on in terms of harmfulness, need for refactoring, and severity level. This work aims to lay the groundwork for creating an automatic test smells detection tool for Python.

Table 1 presents the test smells we analyzed in this study. We selected this subset as these are frequently reported in the literature [15, 18, 27, 30]. In addition, they are among the test smells existing tool support could automatically detect [19, 29].

4 METHODOLOGY

This section presents the research methodology, which comprises two main activities: (i) build a Python test files dataset and (ii) analyze occurrence standard of test smells in Python through the built dataset, looking for test smell occurrences.

To analyze the test smells occurrence standard in Python, we could not consider what already exists in the literature from the ground up. Hence, we first analyzed state-of-the-art approaches and identified that most studies are conducted for Java. Although we do not use a Java dataset in this analysis, we compared the test smells in Python against Java presented in related works. Test smells are language-independent; however, the 10 test smells proposed by Peruma et al. [18], presented in Table 1 and our analysis, were conducted only in Java.

Initially, we created a script to automate the procedures for selecting and collecting Python test files from the GitHub repositories for building the dataset. We used two search tools to select Python project repositories containing test files. The first tool, GitHub Search³, was adopted as a repository selection criteria for projects with at least 50 stars. In the second tool, GitHub Gists⁴, we searched potential users who create test cases using unittest and pytest; for the GitHub Gists, no minimum number of stars was considered.

²https://figshare.com/articles/conference_contribution/Handling_Test_Smells_in_Python_Results_from_a_Mixed-Method_Study/14938650

³<https://github.com/search>

⁴<https://gist.github.com>

Table 1: List of Test Smells investigated in this work.

#	Acronym	Test Smell	Detection Rule	Source
1	AR	Assertion Roulette	Multiple unexplained assertions in the same test method	Deursen et al. [7]
2	CI	Constructor Initialization	Test class with constructor method and field initialization	Peruma et al. [18]
3	CLT	Conditional Logic Test	Test methods that contain conditional statements	Meszaros et al. [17]
4	DA	Duplicate Assert	More than one of the same verification method that checks the same parameters	Peruma et al. [18]
5	DT	Default Test	Automatically generated test class with default name	Peruma et al. [18]
6	EA	Eager Test	Test method that checks many methods of the object to be tested	Deursen et al. [7]
7	EH	Exception Handling	When an exception handling is performed within a test method.	Peruma et al. [18]
8	EM	Empty Test	Test method without executable statements	Peruma et al. [18]
9	GF	General Fixture	Setup method fields that are not used in all test methods	Deursen et al. [7]
10	IT	Ignored Test	When a test method is ignored through the framework's annotation	Peruma et al. [18]
11	LT	Lazy Test	Several test methods that verify the same production method	Deursen et al. [7]
12	MG	Mystery Guest	When a test method uses and depends on external resources	Deursen et al. [7]
13	MNT	Magic Number Test	Test Method that contains some number without indicating their purpose	Meszaros et al. [17]
14	RA	Redundant Assertion	Test method with checks that always have the same response	Peruma et al. [18]
15	RO	Resource Optimism	Test method that optimistically assumes the state of an external resource as available	Deursen et al. [7]
16	RP	Redundant Print	Test method containing screen print statements	Peruma et al. [18]
17	SE	Sensitive Equality	Use toString method in a test method to make validations	Deursen et al. [7]
18	ST	Sleepy Test	Test method with pauses to wait for an event	Peruma et al. [18]
19	UT	Unknown Test	Test method without assertion	Peruma et al. [18]
20	VT	Verbose Test	Test method with many lines of code	Meszaros et al. [17]

The projects in the dataset have an average of 16,363 stars, ranging from 0 to 111k stars, forty projects have less than 50 stars, fifty projects have at least 50 stars, and the dataset could have a wider diversity among the projects because we employed two different tools to select the projects. To filter out the repositories, after collecting the test files we ran a script to go through each collected test file and remove about 300 test files that had no assertion. As a result, we reached 5,303 Python test files, 1,963,184 lines of test code from 90 Python projects [9].

After creating the dataset, the next step was to analyze the dataset looking for test smells in the Python test files. To help this analysis, we developed a script to select samples that might contain certain types of test smells. This script works similarly to the GREP⁵ command. GREP searches for keywords through regular expressions in multiple files. This script returns potential test files with certain test smells. Once returned, we manually parsed them to verify the occurrences from the test carefully smells definitions.

We make all datasets (scripts, forms, answers, code snippets, test smell samples) from this study available [9].

5 TEST SMELLS OCCURRENCE ANALYSIS

This section discusses the test smells occurrence standard analyzed in the Python test files dataset. As earlier mentioned in this paper, Java is the most studied language concerning test smell [1], so we compared the test smells occurrence in Python based on Java.

Out of the 20 test smells analyzed in Python projects, we classified them into four classifications according to their occurrence concerning Java programming: (i) similar occurrence standard, when occurrence standards are almost the same as well Java and Python; (ii) particular aspects, despite having same occurrence standard, there are particular characteristics of programming languages or testing frameworks that slightly modify its occurrence in a test

code; (iii) distinct occurrence standard, having same anomalies impacts, but different occurrence way; and (iv) test smell not found.

In the first one, similar occurrence standard, eight test smells are classified: EM, RA, RP, UT, EA, LT, MNT, MG; in the second one, other eight test smells were placed: AR, DA, CLT, CI, EH, SE, VT, RO; in the third one, three test smells present were classified as distinct occurrence standard: GF, ST, and IT. Finally, we could not find the DT test smell in Python files.

As the first test smell classification had similar occurrence standards, we could use the same detection strategy used in Java. Therefore, we next discuss the test smells placed in the other three classifications, starting with distinct occurrence standard classification.

GF occurs when not all fields in the SetUp method are used in the test methods, making it more generic than necessary. Deursen et al. [7] claim this practice can hinder readability. GF also can impact test suite runtime by doing unnecessary work in this case. It occurs in Java and Python. However, we can avoid it in Python by specifying the fixtures we want to use from the SetUp method in each test method through the `@pytest.mark.usefixtures()` annotation. This way, it is possible to specify, as parameters, fields used in each test method. It is an alternative to avoid unnecessarily allocating fields to all test methods, which can decrease test suite execution time⁶. Using this feature contained in pytest can avoid GF. However, the authors who proposed GF did not opine on whether specifying the fields used in test methods would still indicate a bad practice, compromising readability [7].

ST occurs when a pause in a test method interrupts execution, simulating waiting for an event. This practice can cause unexpected results because processing time may vary in different executions. This practice can occur in Java and Python, but the method used differs. In Python, a feature is used through a parameter in the wait method, called "timeout". Timeout parameter can prevent the test

⁵<https://www.gnu.org/software/grep/manual/grep.html>

⁶<https://docs.pytest.org/en/stable/fixture.html>

suite from taking longer than necessary but does not eliminate test non-determinism. In Java, ST usually occurs with the presence of `thread.sleep`, which has a fixed waiting time. The authors who proposed the ST did not comment on whether using the “smarter” waiting feature would keep such a situation a bad practice [18].

IT occurs when a test method is ignored through the corresponding annotation of the respective framework. The authors of IT believe this practice can hinder test readability and maintainability without aggregating to the test suite. IT differs in Python and Java in the annotation used. In Java, this practice is detected through the `@Ignore` annotation, and using Python, it is possible to ignore a test method wholly or under certain conditions through the `@unittest.skipIf` and `@pytest.mark.skipif` commands. In Java, it is possible to skip a test under certain conditions, but it is not done directly with the `@Ignore` annotation, as IT is detected [19, 29]. The authors who proposed IT did not discuss whether ignoring a test method under certain conditions would remain a bad practice [18].

Regarding our second test smells classification, similar occurrence standard in Java and Python but differences in syntax or language features, AR, DA, and RA differ, e.g., in the list of assertion methods specific to respective test frameworks. On the other hand, CLT and EH differ in structure, and specific keywords programming language, e.g., Java has the `switch` conditional statement that is not contained in Python. In contrast, Python has the `elif` conditional statement that is not contained in Java.

VT occurs when a test method has many lines of code. “Many lines of code” is a relative metric and can differ in different programming languages. The Python language references a leaner syntax, and it is possible to do more operations with fewer lines of code. The number of lines of code for a method to be considered significant may vary depending on the project or programmer’s style. Researchers surveyed Python developers and found that they considered between 29 and 45 lines of code in Python for a method to be considered long [5].

The only test smell not found in the Python test file database was DT. DT occurs when the test class name is generic, for example, “ExampleUnitTest” or “ExampleInstrumentedTest”. The authors who proposed DT explained that this practice is the default name of the test class automatically generated by *Android Studio*⁷, a popular Java mobile development tool, so this test smell is specifically associated with Java. It is recommended that testers rename this test class with an intuitive name of its purpose. This practice had the lowest occurrence among the other test smells [18].

We searched the dataset for test class names with other generic names, such as “Example”, “Instance”, and “Sample”, analogous to the DT identification strategy. However, no test classes were found in Python with these names either. Thus, this evidence may indicate no occurrence of DT in Python.

Therefore, in this work, it can be stated that most of the test smells proposed and discussed apply to Python (19 out of 20). These results may indicate that most of the proposed test smells are programming language-independent, but they may have different occurrences. Consequently, studying detection strategies in other programming languages is required. Furthermore, test smells will

have different detection ways according to each programming language syntax.

6 NEW PROPOSED TEST SMELLS

We noticed occurrences of indicative bad practices in test code during the occurrence analysis of the test smells already proposed and discussed in other programming languages. Therefore, although our analysis used the constructed dataset described in the methodology, we could avoid such practices for better test code quality. We next describe each of the four new proposed test smells, followed by their detection strategy, discussing its significant impacts on test code and possible refactorings.

6.1 Programming Paradigms Blend

Python is a multi-paradigm programming language. The paradigm to be chosen considers the type of system and the goals to achieve. Mixing paradigms in the same project can get confusing. Mixing paradigms within the same test file can get even more confusing. We have found test code examples where developers have mixed the Object-Oriented (OO) and procedural paradigms in the same test code. This practice can make the code hard to understand and maintain.

This practice has already been discussed on programmer platforms^{8 9}. Testing frameworks like JUnit and unittest were designed to cope with the OO paradigm, where test cases are mapped into methods. However, when a code snippet is written following the procedural paradigm, it is interpreted in a global scope and accessed by the whole class, so this practice increases coupling and violates the encapsulation principle of the OO paradigm.

Detection strategy: When there are fields initialized outside the test class (global scope) and used in the test class.
Possible refactorings: Fields located outside the test class can be initialized in the `SetUp` method or in a helper method inside the test class.

6.2 Verifying in Setup Method

Testing frameworks usually have fixture methods called once for all test methods (`setUp` and `TearDown`)¹⁰. These methods work similarly in Python and Java. They contain procedures (such as creating object instances, initializing databases, preparing rules) to execute before and after each test method and intend to help testers make their test code clean, flexible, and make test cases independent of each other.

When the `SetUp` has a check, this method is escaping the single responsibility principle because it is not only initializing data to use in the test cases, but it is also validating something. `SetUp` methods initialize conditions needed for test cases; they are executed once before each test method contained in the test class. When there is a check in an `SetUp` method, it will be processed N times, N being the number of test methods in the test class. This may impact test processing time unnecessarily.

⁸<https://stackoverflow.com/questions/47441017/is-it-bad-practice-to-mix-oo-and-procedural-programming-in-python-or-to-mix-pr>

⁹<https://softwareengineering.stackexchange.com/questions/149578/is-unit-testing-procedural-code-effective>

¹⁰<https://docs.python.org/3/library/unittest.html#class-and-module-fixtures>

⁷<https://developer.android.com>

Detection strategy: When there is an assertion method within the `SetUp` method.

Possible refactorings: Instead of using it in the `SetUp` method that runs before each test method, use it in the `SetUpClass` method that runs only once.

6.3 Non-Functional Statement

When a test method scope becomes empty, it does not execute anything. In Python, this occurs when a scope uses the `pass` keyword. We can also realize it in Java by using the `continue` keyword. This practice generally represents a placeholder for future code. Usually, we can remove this scope without losing functionality. When the `pass` statement executes, nothing happens. However, it saves the developer from getting an error message because an empty scope is not allowed in loops, conditional structures, function definitions, or class definitions. This is a practice to avoid, as it may not add functional value to test code and may make test code more difficult to understand.

Detection strategy: When there is an empty scope within a test method. In Python, this occurs with the use of the `pass` keyword within a scope.

Possible refactorings: Remove the empty scope or implement the expected functionality.

6.4 Undefined Test

By the convention of Python testing frameworks (unittest and pytest), test methods are executed as test cases when they have the prefix “test” in method name¹¹. Otherwise, the method is not executed as a test case. In JUnit, this information is passed to the compiler through the “@Test” annotation. Test annotation is a feature that has been studied for its importance in code. The annotations, besides passing informative instructions to the compiler, help in the understanding, maintenance, and code quality [16]. This practice could be analogous to other programming languages if an annotation is not used as agreed upon by the testing framework.

Detection strategy: It occurs when the name of a test method is not prefixed with “test”.

Possible refactorings: Rename the test method using the prefix “test” or remove it.

7 PRELIMINARY EVALUATION

In this preliminary evaluation, we invited 126 Brazilian Python developers with knowledge in pytest or unittest, and we got 40 participants. We used the LinkedIn search tool to filter these profiles in the platform to achieve this goal. We sent invitation messages directly through the LinkedIn platform with these randomly collected profiles containing the link to our Google Forms questionnaire. It is worth noting that the sample included software development professionals since the target audience was composed of developers and testers.

This questionnaire contained 13 questions, 12 closed-ended questions and one open-ended question. This questionnaire aimed to

assess whether Python developers agree that these four proposed practices are harmful to test code. For each of these practices, they were asked if they believe it can harm test code quality, if they would refactor, how much they think it would impact the test code, on a 0-10 scale, and an open-ended question about the general survey aspects. In the questionnaire, for each type of test smell asked, its definition, potential problems, and ways of detection were presented. We did not mention the term “test smells” as a strategy to reduce respondent bias; we used the term “practice” instead. In addition, no code snippets were presented, as this may interfere with the participants’ answers.

Table 2 presents the results of the 40 developers’ opinion poll regarding the 4 test smells proposed in this paper.

We found the number of developers who believe that such practices are harmful to test code quality, between 72.5% and 95%; those who would refactor, between 65% and 82.5%; and the severity level on a scale from 0 to 10, between 5.83 and 8.20, relevant to the proposed test smells.

Some participants reported that while they agree that the practices are harmful, they believe that the refactoring decision requires further analysis of the project context. Other participants reinforced the need to follow good practices in test code because it is also essential to serving as project documentation, adding value to its maintainability.

A threat to validity concerning test smell detection includes the manual analysis, as it may be susceptible to misunderstanding. To mitigate this, we built a dataset of 5,303 project test files to analyze test smells at different occurrences for each test smells type in real projects. Also, the test smells occurrence standard in Python was only analyzed by the authors of this paper. Different contexts, professionals, and test smells occurrence cases may present different views. However, we based our analyses on the definitions given by the respective authors of each test smell analyzed.

8 CONCLUSION

This paper reports on the analysis of the occurrence standard of 20 test smell in Python projects. To perform this analysis, we built a dataset of 5,303 test files from 90 projects on GitHub. In this analysis, we noticed four patterns of indicative bad practices in Python test code, 3 of which are programming language-independent and 1 for multi-paradigm languages. In addition, we found that some test smells may occur differently in programming languages, so conclusions regarding test smells may require further analysis in other contexts.

This work is part of an ongoing project with the primary goal of developing a test smells detection tool for the Python programming language due to the growing demand and current lack of support for test smells detection in Python.

As future work, we intend to verify whether the test smells proposed in this paper are applicable in other programming languages and how they behave. Also, further research on refactoring strategies for the proposed test smells is necessary.

The possible refactorings indicated in each proposed test smells may vary from case to case and have differences of opinion among developers. Therefore, further analysis with professionals in the

¹¹<https://docs.python.org/3/library/unittest.html>

Table 2: Survey results concerning the new proposed test smells

Test Smell	Harmful (%)			Refactor (%)			Severity level (0-10)		
	Yes	No	Don't know	Yes	No	Don't know	Average	Median	Mode
Programming Paradigms Blend	95.00	5.00	0.00	67.50	17.50	15.00	6.68	7.00	7.00
Verifying in Setup Method	85.00	10.00	5.00	80.00	17.50	2.50	7.28	8.00	9.00
Non-Functional Statement	72.50	20.00	7.50	65.00	27.5	7.50	5.83	6.00	10.00
Undefined Test	87.50	5.00	7.50	82.50	10.00	7.50	8.20	8.50	10.00

field is needed to offer more effective refactoring solutions for such practices.

ACKNOWLEDGMENTS

This research has been partially supported by *Conselho Nacional de Desenvolvimento Científico e Tecnológico* (CNPq) grant 133477/2020-7 and FAPESB grant JCB0060/2016.

REFERENCES

- [1] Wajdi Aljedaani, Anthony Peruma, Ahmed Aljohani, Mazen Alotaibi, Mohamed Wiem Mkaouer, Ali Ouni, Christian Newman, Abdullatif Ghallab, and Stephanie Ludi. 2021. Test Smell Detection Tools: A Systematic Mapping Study. In *International Conference of East-Asian Association for Science Education (EASE)*. Cornell University, Shizuoka, Japan.
- [2] Y. Amannejad, V. Garousi, R. Irving, and Z. Sahaf. 2014. A Search-Based Approach for Cost-Effective Software Test Automation Decision Support and an Industrial Case Study. In *IEEE Seventh International Conference on Software Testing, Verification and Validation Workshops*. IEEE, Cleveland, USA, 302–311.
- [3] Gabriele Bavota, Abdallah Qusef, Rocco Oliveto, Andrea Lucia, and Dave Binkley. 2015. Are Test Smells Really Harmful? An Empirical Study. *Empirical Software Engineering* 20, 4 (2015), 1052–1094.
- [4] Z. Chen, L. Chen, W. Ma, and B. Xu. 2016. Detecting Code Smells in Python Programs. In *International Conference on Software Analysis, Testing and Evolution (SATE)*. IEEE, Kunming, China, 18–23.
- [5] Zhifei Chen, Lin Chen, Wanwangying Ma, Xiaoyu Zhou, Yuming Zhou, and Baowen Xu. 2018. Understanding Metric-Based Detectable Smells in Python Software. *Inf. Softw. Technol.* 94, C (2018), 14–29.
- [6] Jonas De Bleser, Dario Di Nucci, and Coen De Roover. 2019. Assessing diffusion and perception of test smells in scala projects. In *16th International Conference on Mining Software Repositories (MSR)*. IEEE, Montreal, QC, Canada, 457–467.
- [7] Arie Van Deursen, Leon Moonen, Alex Van Den Bergh, and Gerard Kok. 2001. Refactoring test code. In *2nd international conference on extreme programming and flexible processes in software engineering (XP2001)*. CWI (Centre for Mathematics and Computer Science), NLD, 92–95.
- [8] Moritz Eck, Fabio Palomba, Marco Castelluccio, and Alberto Bacchelli. 2019. Understanding Flaky Tests: The Developer's Perspective. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2019)*. ACM, New York, USA, 830–840.
- [9] Daniel Fernandes, Ivan Machado, and Rita Maciel. 2021. Attachments from Handling Test Smells in Python: Results from a Mixed-Method Study. <https://doi.org/10.6084/m9.figshare.14938650>
- [10] Martin Fowler. 1999. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, Boston, MA, USA.
- [11] Vahid Garousi and Barış KUÇUK. 2018. Smells in software test code: A survey of knowledge in industry and academia. *Journal of Systems and Software* 138 (2018), 52–81.
- [12] Dorothy Graham, Erik Van Veenendaal, Isabel Evans, and Rex Black. 2008. *Foundations of Software Testing: ISTQB Certification*. Intl Thomson Business Pr, Cengage Learning Emea.
- [13] Michaela Greiler, Arie van Deursen, and Margaret-Anne D. Storey. 2013. Automated Detection of Test Fixture Strategies and Smells. In *6th IEEE International Conference on Software Testing, Verification and Validation*. IEEE, Luxembourg, 322–331.
- [14] A. Javed, M. Zaman, M. Uddin, and T. Nusrat. 2019. An Analysis on Python Programming Language Demand and Its Recent Trend in Bangladesh. In *Proceedings of the 2019 8th International Conference on Computing and Pattern Recognition (ICCPR '19)*. Association for Computing Machinery, New York, USA, 458–465.
- [15] Nildo Silva Junior, Larissa Rocha Soares, Luana Almeida Martins, and Ivan Machado. 2020. A survey on test practitioners' awareness of test smells. *CoRR* abs/2003.05613 (2020).
- [16] Dong Jae Kim, Nikolaos Tsantalis, Tse-Hsun (Peter) Chen, and Jinqui Yang. 2021. Studying Test Annotation Maintenance in the Wild. In *Proceedings of the 43rd International Conference on Software Engineering*. IEEE, Madrid, ES, 62–73.
- [17] Gerard Meszaros, Shaun M. Smith, and Jennitta Andrea. 2003. The Test Automation Manifesto. In *Extreme Programming and Agile Methods - XP/Agile Universe*, Frank Maurer and Don Wells (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 73–81.
- [18] Anthony Peruma, Khalid Almalki, Christian D. Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2019. On the Distribution of Test Smells in Open Source Android Applications: An Exploratory Study. In *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering (CASCOS '19)*. IBM Corp., USA, 193–202.
- [19] Anthony Peruma, Khalid Almalki, Christian D. Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2020. *TsDetect: An Open Source Test Smells Detection Tool*. ACM, New York, USA, 1650–1654.
- [20] Anthony Peruma, Christian D. Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2020. An Exploratory Study on the Refactoring of Unit Test Files in Android Applications. In *42nd International Conference on Software Engineering Workshops (ICSEW)*. ACM, New York, USA, 350–357.
- [21] Gustavo Pinto, Breno Miranda, Supun Dissanayake, Marcelo d'Amorim, Christoph Treude, and Antonia Bertolino. 2020. What is the Vocabulary of Flaky Tests? In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*. ACM, New York, USA, 492–502.
- [22] G. Van Rossum and F. L. Drake. 2011. *An Introduction to Python*. Network Theory Limited, Network Theory.
- [23] Davide Spadini, Fabio Palomba, A. Zaidman, M. Bruntink, and A. Bacchelli. 2018. On the Relation of Test Smells to Software Code Quality. In *International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, Madrid, Spain, 1–12.
- [24] Davide Spadini, Martin Schvarcacher, Ana-Maria Oprea, Magiel Bruntink, and Alberto Bacchelli. 2020. Investigating Severity Thresholds for Test Smells. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*. ACM, New York, USA, 311–321.
- [25] Suresh Thummalapenta, Saurabh Sinha, Nimit Singhania, and Satish Chandra. 2012. Automating test automation. In *34th International Conference on Software Engineering (ICSE)*. IEEE, Zurich, Switzerland, 881–891.
- [26] F. Trautsch and J. Grabowski. 2017. Are There Any Unit Tests? An Empirical Study on Unit Testing in Open Source Python Projects. In *International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, Tokyo, Japan, 207–218.
- [27] M. Tufano, F. Palomba, G. Bavota, M. Di Penta, R. Oliveto, A. De Lucia, and D. Poshyvanyk. 2016. An empirical investigation into the nature of test smells. In *31st International Conference on Automated Software Engineering (ASE)*. IEEE, Singapore, 4–15.
- [28] Nicole Vavrová and Vadim Zaytsev. 2017. Does Python Smell Like Java? Tool Support for Design Defect Discovery in Python. *The Art, Science, and Engineering of Programming* 1, 2 (2017), 11–29.
- [29] Tássio Virgínio, Luana Martins, Larissa Rocha, Railana Santana, Adriana Cruz, Heitor Costa, and Ivan Machado. 2020. JNose: Java Test Smell Detector. In *34th Brazilian Symposium on Software Engineering (SBES)*. ACM, New York, USA, 564–569.
- [30] Tássio Virgínio, Railana Santana, Luana Almeida Martins, Larissa Rocha Soares, Heitor Costa, and Ivan Machado. 2019. On the influence of test smells on test coverage. In *33rd Brazilian Symposium on Software Engineering*. ACM, New York, USA, 467–471.
- [31] Hongyu Zhai, Casey Casalnuovo, and Prem Devanbu. 2019. Test Coverage in Python Programs. In *Proceedings of the 16th International Conference on Mining Software Repositories*. IEEE Press, Montreal, Canada, 116–120.